

**PE-U4: Procesamiento Distribuido con Apache Spark**  
**Comprobación Experimental de la Ley de Amdahl**  
**sobre un Pipeline de Tickets de Soporte Técnico ISP (ACC)**

**Código de actividad:** GA-SUM-05 / PE-U4  
**Asignatura:** Aplicaciones Distribuidas (ISR-701)  
**Unidad:** Unidad 4  
**Nivel / Paralelo:** Séptimo de Ingeniería de Software “A”  
**Docente:** Gleiston C. Guerrero-Ulloa, M.Sc.  
**Período académico:** 2026–2027 PPA

**PFC de referencia:** ACC — Sistema de Gestión de Soporte Técnico ISP

**Integrantes**

| Nombre                            | PFC de origen | Rol asumido en esta práctica                                   |
|-----------------------------------|---------------|----------------------------------------------------------------|
| Cristhian Daniel Pacheco Cárdenas | ACC           | Implementación PySpark, protocolo de medición y escalado de T3 |
| Robinson Rodrigo Cando Moreno     | ACC           | Análisis de Amdahl, equivalencia y umbral de rentabilidad      |
| Ernesto Gregory Luna Mora         | BCEL          | Repositorio, reproducibilidad, figuras y verificación final    |

**Justificación técnica.** El dominio ACC (Sistema de Gestión de Soporte Técnico ISP) opera sobre un flujo continuo de tickets de soporte –cortes de servicio, reclamos de velocidad, fallas de equipo, solicitudes de instalación y consultas de facturación– generado las 24 horas por una base de clientes activa. Incluso un proveedor de tamaño medio, con algunos miles de clientes, acumula en pocos meses de operación cientos de miles de registros de tickets, cada uno con múltiples atributos temporales, categóricos y de estado. A esa escala, el análisis exploratorio, los reportes gerenciales recurrentes y las agregaciones por zona o categoría dejan de ser viables con herramientas de una sola máquina: el tiempo de cómputo crece de forma que compromete la oportunidad de la decisión operativa. Este trabajo usa exactamente ese volumen (600 000 tickets sintéticos, semilla fija) para comprobar, con datos propios, si un motor distribuido como Apache Spark ofrece una ventaja medible frente a pandas. La decisión de negocio concreta que se apoyaría en este pipeline es identificar qué categorías de incidencia concentran más tickets críticos por zona operativa, para decidir dónde reforzar personal técnico de campo o priorizar mantenimiento de infraestructura de red –una decisión operativa medible, no una mejora genérica del servicio.

# Índice

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                                     | <b>2</b>  |
| <b>1. Introducción</b>                                                             | <b>3</b>  |
| <b>2. Fundamento Teórico</b>                                                       | <b>3</b>  |
| 2.1. Fundamentos de paralelismo y Ley de Amdahl . . . . .                          | 3         |
| 2.2. Paradigma MapReduce y Apache Hadoop . . . . .                                 | 6         |
| 2.3. Apache Spark: arquitectura y modelo de programación . . . . .                 | 7         |
| 2.4. Computación en la nube: modelos y proveedores . . . . .                       | 8         |
| <b>3. Procedimiento Aplicado</b>                                                   | <b>9</b>  |
| 3.1. Entorno y conjunto de datos . . . . .                                         | 9         |
| 3.2. Configuración efectiva de Spark . . . . .                                     | 9         |
| 3.3. Implementación con pandas y PySpark . . . . .                                 | 10        |
| 3.4. Verificación de equivalencia pandas $\leftrightarrow$ PySpark . . . . .       | 11        |
| 3.5. Versiones declaradas del entorno . . . . .                                    | 11        |
| <b>4. Resultados</b>                                                               | <b>12</b> |
| 4.1. Tiempos de ejecución: pandas vs. PySpark . . . . .                            | 12        |
| <b>5. Análisis Crítico</b>                                                         | <b>15</b> |
| 5.1. Speedup de las cinco transformaciones . . . . .                               | 15        |
| 5.2. Fracción serial observada de T3: sustitución numérica . . . . .               | 15        |
| 5.3. Speedup máximo teórico ( $S_{\text{máx}}$ ) . . . . .                         | 16        |
| 5.4. Fracción serial algorítmica vs. sobrecarga observada, aplicada a T3 . . . . . | 16        |
| 5.5. Umbral de rentabilidad de Spark para ACC . . . . .                            | 17        |
| <b>6. Preguntas de Análisis (Anexo A)</b>                                          | <b>17</b> |
| <b>7. Conclusiones Individuales</b>                                                | <b>19</b> |
| <b>Declaración de Uso de Inteligencia Artificial Generativa</b>                    | <b>20</b> |

## Resumen

La Ley de Amdahl (1967) establece el límite teórico de aceleración que puede obtenerse al paralelizar un programa cuya fracción serial es fija, y sigue siendo el marco de referencia para evaluar si motores modernos de procesamiento distribuido, como Apache Spark, cumplen su promesa de escalabilidad sobre cargas de trabajo reales. Este informe comprueba experimentalmente dicha ley aplicando un pipeline de cinco transformaciones (filtrado, agrupación, join, generación de variables derivadas y top-N) sobre un dataset sintético determinista de 600 000 tickets de soporte técnico ISP del dominio ACC, implementado en paralelo en pandas y en PySpark 3.5.3, bajo un protocolo de medición riguroso (calentamiento descartado, cinco repeticiones cronometradas con `time.perf_counter`, estadístico mediana) y con escalado de la transformación de join (T3) en  $N = 1, 2, 4$  executors. Los resultados muestran una aceleración de PySpark frente a pandas de entre  $1.43\times$  y  $2.56\times$  según la transformación, con equivalencia numérica exacta entre ambos motores en las cinco transformaciones (cardinalidad, columnas, sumas, medias y claves de agrupación). El análisis de Amdahl sobre T3 estima una fracción paralelizable observada de apenas 17.3 %, lo que fija un techo teórico de aceleración de  $1.21\times$  y explica por qué el rendimiento se satura ya con  $N = 2$  executors –un hallazgo consistente con el *shuffle* del join como cuello de botella dominante, más que con una limitación algorítmica intrínseca. Estos resultados tienen una implicación directa para ACC: la migración a Spark se justifica por el volumen de datos que hay que procesar, no por el paralelismo bruto de hardware adicional.

# 1. Introducción

El crecimiento sostenido del volumen de datos operativos en sistemas empresariales ha convertido el procesamiento distribuido en una necesidad práctica más que en una opción de diseño: cuando un conjunto de datos supera lo que una sola máquina puede procesar en un tiempo aceptable, la única vía para mantener la oportunidad de las decisiones basadas en esos datos es repartir el cómputo entre varias unidades de procesamiento [1, 2].

Sin embargo, repartir el trabajo entre más procesadores no garantiza una aceleración proporcional. La Ley de Amdahl, formulada en 1967, es el marco clásico que cuantifica ese límite: toda tarea computacional tiene, en la práctica, una fracción que no puede paralelizarse –ya sea por dependencias de datos, coordinación entre nodos o simple lógica secuencial– y esa fracción fija un techo de aceleración que ningún número adicional de procesadores puede superar. Más de cinco décadas después de su formulación, la vigencia de esta ley sigue siendo objeto de verificación empírica cuando se aplica a motores modernos de procesamiento distribuido como Apache Spark, cuyo modelo de ejecución en memoria y evaluación perezosa difiere sustancialmente de los sistemas para los que Amdahl escribió originalmente su argumento.

El objetivo de este trabajo es comprobar experimentalmente la Ley de Amdahl –y su complemento, la Ley de Gustafson-Barsis– aplicando un pipeline real de cinco transformaciones de datos sobre un dataset sintético de 600 000 tickets de soporte técnico ISP, correspondiente al dominio ACC (Sistema de Gestión de Soporte Técnico ISP), implementado en paralelo en pandas (línea base de una sola máquina) y en PySpark (motor distribuido), con escalado controlado de una de las transformaciones en  $N = 1, 2, 4$  executors.

El documento se organiza de la siguiente manera: la Sección 2 desarrolla el fundamento teórico del paralelismo, MapReduce/Hadoop, la arquitectura de Spark y los modelos de computación en la nube; la Sección 3 documenta el procedimiento aplicado (dataset, configuración de Spark, implementación y verificación de equivalencia); la Sección 4 presenta los resultados cuantitativos; la Sección 5 desarrolla el análisis crítico de la fracción serial observada y el umbral de rentabilidad; la Sección 6 responde las preguntas de análisis del Anexo A; y el documento cierra con las conclusiones individuales del equipo y la declaración de uso de inteligencia artificial generativa.

## 2. Fundamento Teórico

### 2.1. Fundamentos de paralelismo y Ley de Amdahl

El paralelismo de cómputo se clasifica, de forma general, en dos estrategias complementarias. El **paralelismo de datos** consiste en dividir un mismo conjunto de datos en fragmentos independientes y aplicar *la misma* operación a cada fragmento de forma simultánea en distintas unidades de procesamiento; es el modelo que sigue Apache Spark al particionar un DataFrame entre executors, donde cada partición se procesa con idéntica lógica de transformación [3, 4]. El **paralelismo de tareas**, en cambio, consiste en ejecutar simultáneamente operaciones *distintas* entre sí, cada una potencialmente sobre datos diferentes, coordinadas para producir un resultado común; es el modelo más cercano a las fases Map y Reduce como unidades de trabajo diferenciadas dentro de un mismo job [1, 5]. En el pipeline de este proyecto, T1–T5 se ejecutan como tareas independientes entre sí (paralelismo de tareas a nivel de pipeline), mientras que, dentro de cada transformación, el procesamiento de las 600 000 filas se reparte por partición entre executors (paralelismo de datos a nivel de transformación).

**Derivación de la Ley de Amdahl.** Sea  $T$  el tiempo total de ejecución de un programa en una sola unidad de procesamiento, normalizado a  $T = 1$ . Este tiempo se compone de dos fracciones complementarias: una fracción serial ( $1 - p$ ), correspondiente a las instrucciones que deben ejecutarse estrictamente en orden (por dependencias de datos, inicialización, o agregación final de resultados) y que ninguna cantidad de procesadores adicionales puede acelerar; y una

fracción paralelizable  $p$ , que sí puede repartirse entre  $N$  unidades de procesamiento sin pérdida de eficiencia ideal. Bajo el supuesto de que la fracción paralelizable se reparte de forma perfectamente uniforme entre las  $N$  unidades, el tiempo que tarda esa porción se reduce a  $p/N$ , mientras que el tiempo de la fracción serial permanece constante en  $(1-p)$ , porque por definición no se distribuye. El tiempo total con  $N$  procesadores es entonces  $(1-p) + p/N$ , y el *speedup*  $S(N)$  –la razón entre el tiempo original y el tiempo paralelizado– resulta de dividir el tiempo original ( $T = 1$ ) entre ese tiempo reducido, lo que da la ecuación (1), conocida como la Ley de Amdahl [6, 7]:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}} \quad (1)$$

**Speedup máximo teórico.** Cuando  $N$  crece sin límite, el término  $p/N$  tiende a 0 y el speedup se estabiliza en el valor dado por la ecuación (2): el speedup máximo teórico depende exclusivamente de la fracción serial, no del número de procesadores disponibles.

$$S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p} \quad (2)$$

La implicación práctica es contundente: si el 10 % de un programa es estrictamente serial ( $p = 0,9$ ), el speedup máximo posible es  $1/(1-0,9) = 1/0,1 = 10$ , sin importar si se usan 100 o 10 000 procesadores –agregar más recursos de cómputo más allá de cierto punto deja de tener retorno [6, 7].

**Eficiencia del paralelismo.** La eficiencia, definida en la ecuación (3), mide qué fracción del beneficio ideal (speedup lineal,  $E = 1$ ) se aprovecha realmente con cada unidad de procesamiento adicional.

$$E(N) = \frac{S(N)}{N} \quad (3)$$

Un speedup absoluto alto obtenido con un número grande de procesadores puede, pese a todo, tener una eficiencia baja: es la señal cuantitativa de que se está entrando en la zona de rendimientos decrecientes que predice la propia Ley de Amdahl [6, 4].

**Ley de Gustafson-Barsis.** La Ley de Amdahl asume un supuesto que rara vez se sostiene en sistemas de Big Data: que el tamaño del problema es fijo y lo único que cambia es el número de procesadores disponibles para resolverlo más rápido. Gustafson [7] invierte ese supuesto en 1988: en la práctica, cuando una organización agrega más capacidad de cómputo no es porque quiera resolver exactamente el mismo problema en menos tiempo, sino porque quiere procesar más datos en el mismo tiempo disponible. Bajo ese supuesto de carga de trabajo escalable, el speedup observado se modela con la ecuación (4), conocida como Ley de Gustafson-Barsis:

$$S_G(N) = N - (1-p)(N-1) \quad (4)$$

Esta distinción es particularmente relevante para sistemas como Apache Spark: cuando un equipo de datos escala un clúster, típicamente no es para terminar el mismo reporte más rápido, sino para poder procesar, en la misma ventana de tiempo, un volumen de tickets, transacciones o eventos que ha crecido [7, 2]. Por esta razón, las leyes de Amdahl y Gustafson-Barsis no son contradictorias sino complementarias: la primera acota el beneficio de paralelizar un problema de tamaño fijo, y la segunda describe el escenario –mucho más común en la práctica de Big Data– de un problema cuyo tamaño crece junto con los recursos.

La Figura 1 ilustra las curvas de speedup predichas por la ecuación (1), mostrando cómo el techo de aceleración se desplaza drásticamente según qué tan paralelizable sea, en términos algorítmicos, el problema: incluso con  $p = 0,95$  (un programa con solo 5 % de código estrictamente serial), el speedup se satura por debajo de  $20\times$ , mientras que con  $p = 0,50$  el techo es de apenas  $2\times$ .

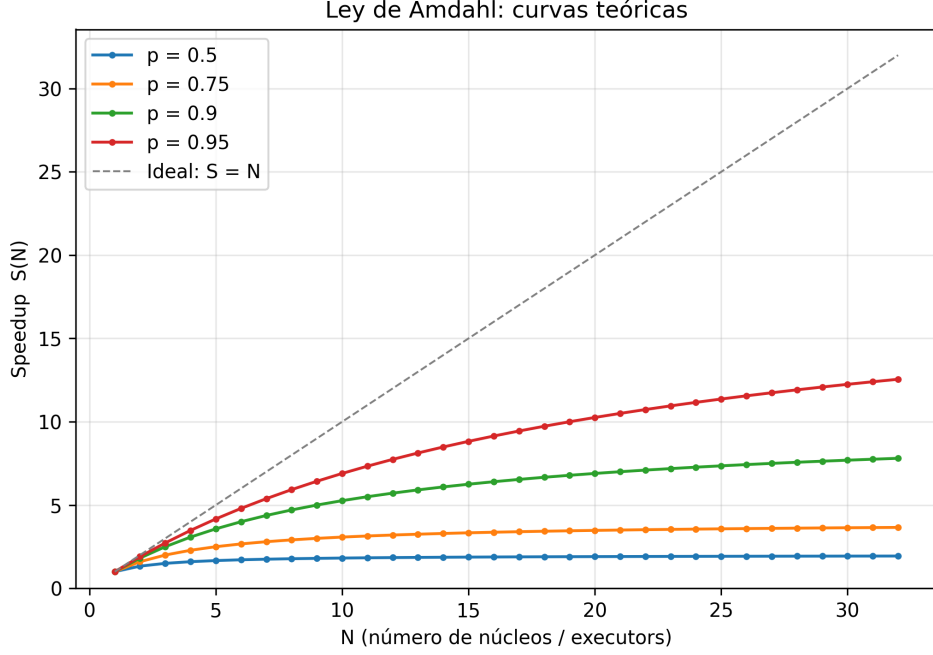


Figura 1: Curvas teóricas de speedup  $S(N)$  según la ecuación (1) para cuatro valores de la fracción paralelizable ( $p = 0,50, 0,75, 0,90, 0,95$ ).

**Fracción serial observada (forma inversa).** Despejando  $p$  de la ecuación (1) en función de un speedup y un  $N$  observados experimentalmente, se obtiene la forma inversa de la ecuación (5), que permite estimar –a partir de mediciones reales, no de una declaración de diseño– qué fracción del tiempo total se comportó como si fuera serial en una ejecución concreta:

$$f_{\text{obs}}(N) = \frac{\frac{1}{S(N)} - \frac{1}{N}}{1 - \frac{1}{N}} \quad (5)$$

Es indispensable, sin embargo, no confundir esta magnitud despejada con la fracción serial algorítmica del programa:  $f_{\text{obs}}(N)$  no distingue entre instrucciones que son intrínsecamente secuenciales (por diseño del algoritmo) y tiempo perdido por sobrecarga de planificación de tareas, serialización de datos entre procesos o –de forma especialmente relevante en Spark– el *shuffle* necesario para redistribuir datos entre executors en operaciones como el join o el **groupBy**. Confundir la fracción serial observada con la fracción serial algorítmica es un error conceptual explícitamente penalizado en el criterio 2.2 de esta rúbrica, y se retoma de forma aplicada, con los datos reales del experimento, en la Sección 5.

**Conexión con el dominio ACC.** En el pipeline de tickets de soporte de ACC, esta distinción tiene una traducción directa: T1 (filtrado) es una operación de paralelismo de datos casi puro –cada fila se evalúa de forma completamente independiente contra la máscara de condiciones, sin que el resultado de un ticket dependa del de otro– por lo que se espera que su fracción serial observada sea baja. T3 (join), en cambio, requiere redistribuir físicamente las filas de **tickets** y **agentes** entre executors según la clave **agente\_id** antes de poder emparejarlas (*shuffle*), una operación de comunicación entre nodos que Amdahl, en su formulación clásica de 1967, no distingue de la fracción serial pura del algoritmo. Por esta razón, T3 es la transformación elegida en este experimento para el escalado con  $N = 1, 2, 4$  executors: es la que mejor expone, con datos reales, la diferencia entre paralelismo teórico y paralelismo efectivo.

## 2.2. Paradigma MapReduce y Apache Hadoop

El modelo de programación MapReduce, formalizado por Dean y Ghemawat en Google y publicado en su versión ampliamente citada en 2008 [1], surgió de una motivación concreta: permitir que ingenieros sin experiencia en sistemas distribuidos pudieran procesar volúmenes de datos del orden de terabytes o petabytes sobre clústeres de hardware común (*commodity hardware*), abstrayendo de la lógica de la aplicación los detalles de distribución de datos, paralelización, tolerancia a fallos y balanceo de carga [1, 5].

El modelo se estructura en dos funciones puras que el programador debe implementar. La función **Map** recibe pares clave-valor de entrada y produce, para cada uno, cero o más pares clave-valor intermedios. La función **Reduce** recibe, para cada clave intermedia distinta, la lista completa de valores asociados a esa clave (ya agrupados por el propio framework) y produce el resultado final agregado para esa clave [1]. Un ejemplo aplicado al dominio ACC ilustra el patrón con claridad: para contar tickets por categoría de incidencia, Map emitiría, por cada ticket procesado, el par (`categoría`, 1); Reduce recibiría, para cada categoría, la lista completa de unos emitidos y devolvería su suma —exactamente la agregación de conteo que en este proyecto se implementa como T2 mediante `groupBy`.

La ejecución completa de un job de MapReduce atraviesa cinco fases [1, 5]: *split*, donde el dataset de entrada se divide en fragmentos independientes que pueden procesarse en paralelo; *map*, donde cada fragmento se procesa de forma independiente y en paralelo por una tarea Map; *shuffle*, donde los pares intermedios se redistribuyen entre los nodos según su clave, de modo que todos los valores de una misma clave terminen en el mismo nodo; *sort*, donde los valores dentro de cada partición se ordenan por clave antes de agruparlos; y *reduce*, donde se ejecuta la función de agregación final sobre cada grupo de clave-valores.

La implementación de referencia de este modelo, Apache Hadoop, se apoya en dos componentes arquitectónicos: HDFS (*Hadoop Distributed File System*), un sistema de archivos distribuido que replica cada bloque de datos en varios nodos (típicamente tres copias) para garantizar tolerancia a fallos sin depender de hardware especializado; y YARN (*Yet Another Resource Negotiator*), el gestor de recursos del clúster que asigna memoria y cómputo a las tareas Map y Reduce en ejecución [5, 1]. Esta arquitectura ofrece dos ventajas centrales: escalabilidad horizontal con hardware económico (agregar más nodos aumenta linealmente la capacidad de almacenamiento y cómputo) y tolerancia a fallos automática mediante replicación, sin intervención del programador [5].

Sin embargo, el propio diseño de MapReduce impone una limitación estructural relevante para cargas de trabajo iterativas o de consulta repetida sobre el mismo conjunto de datos: cada job MapReduce lee sus datos de entrada desde disco (HDFS) al iniciar y escribe su resultado de vuelta a disco al terminar, incluso si el siguiente job en una cadena de procesamiento va a volver a leer exactamente esos mismos datos. En algoritmos iterativos —consultas repetidas, algoritmos de aprendizaje automático que requieren múltiples pasadas sobre el mismo dataset— este patrón genera una sobrecarga de entrada/salida a disco que se repite en cada iteración, en vez de aprovechar que los datos ya están disponibles en memoria desde la iteración anterior. Esta limitación es, precisamente, el problema central que Apache Spark fue diseñado para resolver mediante su modelo de datos en memoria, desarrollado en la Sección 2.3.

Por tratarse de una comparación de rendimiento entre motores, cualquier cifra concreta de cuántas veces más rápido es un motor respecto al otro debe respaldarse con evidencia empírica publicada y no con material promocional de proveedores. El estudio de referencia usado en este informe para esa comparación es el benchmark HiBench de Ahmed et al. [8], retomado con datos propios en la Sección 2.3.



### 2.3. Apache Spark: arquitectura y modelo de programación

Apache Spark organiza su ejecución en tres componentes [2, 4]: el **driver**, el proceso donde se ejecuta el programa principal del usuario, se construye el grafo de transformaciones y se coordina la ejecución distribuida; el **cluster manager** (YARN, Kubernetes, Mesos, o el modo `local[N]` usado en este proyecto), responsable de asignar recursos físicos a la aplicación; y los **executors**, procesos que se ejecutan en los nodos de trabajo, corren las tareas asignadas y almacenan particiones de datos en memoria o disco según se requiera. En el modo `local[N]` usado en Colab para este experimento, todos estos roles conviven en una sola máquina, y  $N$  determina el número de hilos que simulan executors –una simplificación necesaria para un entorno educativo de una sola máquina, y una limitación que se documenta explícitamente en la Sección 3.

La abstracción fundamental de datos de Spark es el RDD (*Resilient Distributed Dataset*), introducido por Zaharia et al. [3]: una colección distribuida e inmutable de objetos, particionada entre los nodos del clúster, sobre la que solo pueden aplicarse transformaciones deterministas. La propiedad clave de los RDD para la tolerancia a fallos es su *lineage* (linaje): en vez de replicar físicamente los datos como hace HDFS, Spark registra la secuencia de transformaciones que produjo cada RDD a partir de los datos originales. Si una partición se pierde por el fallo de un nodo, Spark puede reconstruirla recalculando únicamente esa porción del linaje, sin necesidad de leer una copia física de respaldo [3]. Esta es una diferencia conceptual central frente al modelo de HDFS descrito en la Sección 2.2.

Sobre los RDD, Spark ofrece capas de abstracción de más alto nivel –DataFrames y Datasets– que añaden un esquema estructurado (columnas con nombre y tipo declarado), de forma análoga a una tabla relacional [2, 4]. Este esquema explícito es lo que permite al motor aplicar optimizaciones automáticas que sobre un RDD sin tipado estructurado no serían posibles de forma genérica. Todas las transformaciones T1–T5 de este proyecto están implementadas sobre la API de DataFrames, tanto en su versión PySpark como en su equivalente conceptual de pandas.

Un rasgo de diseño central de Spark, y que tuvo una implicación metodológica directa en este experimento, es la **evaluación perezosa** (*lazy evaluation*): las transformaciones (`filter`, `groupBy`, `join`, `withColumn`) no se ejecutan en el momento en que se invocan en el código; Spark las acumula en un plan lógico y, sobre él, construye un plan físico optimizado, pero no materializa ningún cómputo hasta que se invoca una **acción** (`count()`, `collect()`, o una escritura a disco como `write.csv()`) [2, 4]. Esta característica es la que obliga, técnicamente, a que el protocolo de medición de este proyecto (Sección 3) fuerce siempre una escritura a disco al final de cada transformación medida: sin una acción explícita, un cronómetro colocado alrededor del código de transformación mediría solo el tiempo de construir el plan de ejecución (microsegundos), no el trabajo real de leer, transformar y persistir 600 000 filas.

El plan físico que Spark construye se representa como un DAG (grafo acíclico dirigido) de *stages*, y cada stage se subdivide en *tasks* que se ejecutan en paralelo sobre las particiones de datos, típicamente uno por núcleo/executor disponible [2]. Este es exactamente el nivel de detalle que se observa en la captura de la Spark UI incluida como evidencia en la Sección 4 (Figura 5), que muestra el DAG real de la transformación T3 (`join`) ejecutándose sobre el clúster local de Colab, incluyendo la etapa de *shuffle* discutida en la Sección 5.

Como comparación de rendimiento frente a MapReduce: el procesamiento en memoria de Spark evita, en cargas de trabajo que reutilizan datos entre operaciones, las escrituras a disco entre etapas que penalizan a MapReduce en flujos iterativos (Sección 2.2); el benchmark HiBench de Ahmed et al. [8] documenta empíricamente esta ventaja sobre cargas de trabajo comparables, resultado que se contrasta con los datos propios de este proyecto en la Sección 4.

Como complemento recomendado, vale precisar el papel del optimizador **Catalyst**: antes de ejecutar un plan sobre DataFrames, Spark lo reescribe automáticamente –por ejemplo, reordenando los filtros para que se apliquen lo antes posible y reduzcan cuanto antes el volumen de datos que atraviesa las etapas siguientes– lo que explica por qué las transformaciones sobre DataFrames no necesariamente se ejecutan en el orden textual en que fueron escritas en el código



[2, 4].

## 2.4. Computación en la nube: modelos y proveedores

El Instituto Nacional de Estándares y Tecnología de Estados Unidos (NIST) define el cómputo en la nube, en su publicación de referencia SP 800-145, como un modelo que habilita acceso bajo demanda, conveniente y a través de la red, a un conjunto compartido de recursos de cómputo configurables (redes, servidores, almacenamiento, aplicaciones y servicios) que pueden aprovisionarse y liberarse con un esfuerzo de gestión mínimo [9]. Esta definición formaliza cuatro modelos de servicio, organizados por el nivel de abstracción que ofrecen al usuario: **IaaS** (*Infrastructure as a Service*), que expone máquinas virtuales, redes y almacenamiento en bruto; **PaaS** (*Platform as a Service*), que entrega un entorno de ejecución gestionado sobre el cual el usuario despliega su propia aplicación sin administrar la infraestructura subyacente; **SaaS** (*Software as a Service*), donde el usuario consume una aplicación completa a través de la red; y **FaaS** (*Function as a Service*), un modelo de cómputo *serverless* en el que el usuario despliega funciones individuales que se ejecutan en respuesta a eventos, sin aprovisionar servidores de forma explícita [9].

El mismo documento del NIST identifica cinco características esenciales que distinguen al cómputo en la nube de la infraestructura tradicional [9]: *autoservicio bajo demanda* (el usuario aprovisiona recursos de cómputo automáticamente, sin intervención humana del proveedor); *acceso amplio a la red* (los recursos están disponibles a través de mecanismos estándar, accesibles desde plataformas heterogéneas); *agrupación de recursos* (*pooling*), donde los recursos físicos del proveedor sirven a múltiples clientes bajo un modelo multi-inquilino, asignados dinámicamente según demanda; *elasticidad rápida*, la capacidad de aprovisionar y liberar recursos de forma prácticamente instantánea para adaptarse a la carga; y *servicio medido*, donde el uso de recursos se monitorea, controla y factura de forma transparente para el proveedor y el consumidor.

Los tres proveedores dominantes de nube pública –Amazon Web Services, Microsoft Azure y Google Cloud Platform– ofrecen cada uno un servicio gestionado específicamente para ejecutar Apache Spark a escala de producción: Amazon EMR, Azure HDInsight (o su sucesor Azure Databricks) y Google Cloud Dataproc, respectivamente [4, 2]. Estos servicios abstraen exactamente la parte de la gestión de infraestructura que, en este proyecto, se simplificó artificialmente con fines educativos usando el modo `local[N]` de Google Colab: el aprovisionamiento de nodos, el escalado automático del clúster según la carga, el parcheo de seguridad del sistema operativo y la configuración de red entre executors. En un despliegue real de un sistema de soporte técnico ISP como ACC, sería uno de estos tres servicios gestionados –no un clúster local simulado– el que sostendría el pipeline de procesamiento en producción.

Finalmente, dos patrones arquitectónicos son relevantes para decidir cómo evolucionaría este pipeline si los datos de tickets llegaran en tiempo real en vez de en lote, como se discute en la Pregunta 3 del Anexo A (Sección 6): la arquitectura **Lambda** combina una capa de procesamiento por lotes (*batch*, que recalcula vistas exactas sobre el histórico completo de datos, con alta latencia pero exactitud garantizada) con una capa de velocidad (*speed layer*, que procesa el flujo reciente de eventos con baja latencia pero resultados aproximados), unificadas en una capa de servicio que combina ambas vistas para responder consultas; la arquitectura **Kappa**, por su parte, simplifica este esquema eliminando la capa de lotes y tratando también los datos históricos como un flujo reproducible sobre el mismo motor de streaming, evitando así mantener dos rutas de código distintas para el mismo cómputo [2, 9].

Si el pipeline de tickets de soporte de ACC se llevara a producción, la elección entre procesamiento por lotes (el enfoque de esta práctica) y streaming dependería directamente de la urgencia de la decisión de negocio que dependa de él: un reporte gerencial semanal de tickets por categoría admite perfectamente un procesamiento batch nocturno, pero una alerta de corte masivo de servicio –donde cientos de tickets de la misma zona llegan en minutos– exige una arquitectura de streaming (Lambda o Kappa) capaz de disparar una respuesta operativa casi en tiempo real.

### 3. Procedimiento Aplicado

#### 3.1. Entorno y conjunto de datos

No existe un dataset real público de tickets de soporte ISP con al menos 500 000 registros (verificado por el equipo). La guía de la asignatura permite explícitamente el uso de datos sintéticos generados con **Faker** y semilla fija, declarado en el informe. Por ello, el equipo generó de forma determinista (`src/generar_dataset_acc.py`, semilla 42) dos tablas relacionadas mediante la clave foránea `tickets.agente_id`  $\rightarrow$  `agentes.agente_id`, diseño que hace genuino el *join* de T3 (no un *self-join* sobre el propio dataset). El esquema (categorías, estados, prioridades con su SLA en horas, canales y zonas) está anclado al dominio real del PFC ACC. La Tabla 1 resume los metadatos exigidos por el criterio 1.1.

Cuadro 1: Metadatos del dataset sintético de tickets ISP (ACC).

| Campo                     | Valor                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------|
| Fuente                    | Sintético (Faker <code>es_MX</code> + numpy/pandas, semilla fija 42)                    |
| URL (generación)          | <code>src/generar_dataset_acc.py</code> (repositorio Soporte-Tecnico-ACC)               |
| Licencia                  | Datos generados por el propio equipo; código bajo licencia MIT ( <code>LICENSE</code> ) |
| Fecha de generación (UTC) | 2026-08-07T04:50:09+00:00                                                               |
| Registros                 | 600 000 tickets / 300 agentes                                                           |
| Columnas                  | 15 ( <code>tickets.csv</code> ) / 7 ( <code>agentes.csv</code> )                        |
| Tamaño en disco           | 75.8 MB ( <code>tickets.csv</code> ) / <0.1 MB ( <code>agentes.csv</code> )             |

Los valores ausentes se serializan como cadena vacía en el CSV (`agente_id` vacío: ticket sin agente asignado; `fecha_resolucion` vacía: ticket aún abierto). pandas convierte  $\rightarrow$  NaN y PySpark  $\rightarrow$  NULL: ese criterio único de normalización de nulos, aplicado idénticamente en ambos motores, es lo que garantiza la equivalencia numérica de T1–T5 verificada en la Sección 3.4 (criterio 1.4). `data/raw/` no se versiona en el repositorio (es regenerable bit a bit con los mismos argumentos); en su lugar se versionan los diez CSV de salida del pipeline (`data/pandas/*.csv`, `data/spark/*.csv`) como evidencia directa de la ejecución.

#### 3.2. Configuración efectiva de Spark

La sesión de Spark se inicializa con `master=local[N]`, donde  $N \in \{1, 2, 4\}$  simula el número de executors en una sola máquina (Google Colab). El parámetro `spark.executor.instances=4` se fija por requisito del checklist de la asignatura, pero en modo `local[N]` Spark crea siempre *un* proceso executor con  $N$  hilos: la paralelización efectiva es la que da `local[N]`, no el valor de `executor.instances`, limitación que se documenta explícitamente para que no se interprete erróneamente como un clúster de cuatro nodos físicos. El *broadcast join* se desactiva (`autoBroadcastJoinThreshold=-1`) para forzar que T3 ejecute siempre un *sort-merge join* con *shuffle* real y distribuido; sin esta configuración, Spark transmitiría la tabla pequeña `agentes` a todos los executors y el *shuffle* –el fenómeno central del análisis de la Sección 5– desaparecería. La Tabla 2 transcribe, sin modificaciones, el subconjunto relevante de la salida real de `spark.sparkContext.getConf().getAll()` obtenida en la ejecución documentada en `notebooks/PE_U4_pipeline_spark.ipynb`.

El número de particiones de *shuffle* se fija en `max(4×N, 8)` (16 para  $N = 4$ ), de modo que el paralelismo del *shuffle* escale acorde al número de executors simulados en cada corrida. `spark.sql.session.timeZone=UTC` asegura que las funciones `year()/month()` de Spark extraí-

Cuadro 2: Configuración efectiva de la SparkSession (salida real de `getConf().getAll()`, ejecución N=4).

| Clave                                             | Valor                             |
|---------------------------------------------------|-----------------------------------|
| <code>spark.master</code>                         | <code>local[4]</code>             |
| <code>spark.app.name</code>                       | <code>pe-u4-notebook (N=4)</code> |
| <code>spark.executor.instances</code>             | <code>4</code>                    |
| <code>spark.sql.autoBroadcastJoinThreshold</code> | <code>-1</code>                   |
| <code>spark.sql.shuffle.partitions</code>         | <code>16</code>                   |
| <code>spark.sql.session.timeZone</code>           | <code>UTC</code>                  |
| <code>spark.ui.enabled</code>                     | <code>true</code>                 |
| <code>spark.driver.host</code>                    | <code>127.0.0.1</code>            |

gan los mismos componentes de fecha que `pandas.Timestamp` en T4. `spark.ui.enabled=true` habilita la Spark UI en el puerto 4040, capturada como evidencia en la Figura 5.

### 3.3. Implementación con pandas y PySpark

Las cinco transformaciones (T1 filtrado, T2 agrupación, T3 join, T4 derivadas, T5 top-N) se implementan de forma espejada en los módulos `transformaciones_pandas.py` y `transformaciones_spark.py`, dentro de la carpeta `src/`. A continuación se listan los fragmentos representativos de cada transformación; el código completo, incluyendo la lectura y normalización de nulos, está disponible en el repositorio.

```

1 def t1_filtrado(df: pd.DataFrame) -> pd.DataFrame:
2     return base_filtrada(df)[
3         ["ticket_id", "cliente_id", "agente_id", "categoria", "prioridad",
4          "canal", "zona", "fecha_creacion", "fecha_resolucion"]
5     ]

```

Listing 1: T1 – Filtrado (pandas).

```

1 def t1_filtrado(df):
2     return base_filtrada(df).select(
3         "ticket_id", "cliente_id", "agente_id", "categoria", "prioridad",
4         "canal", "zona", "fecha_creacion", "fecha_resolucion",
5     )

```

Listing 2: T1 – Filtrado (PySpark).

```

1 def t2_agrupacion(df: pd.DataFrame) -> pd.DataFrame:
2     d = df[df["categoria"].notna() & df["zona"].notna()].copy()
3     g = (
4         d.groupby(["categoria", "zona"], dropna=False)
5         .agg(
6             cnt=("ticket_id", "count"),
7             sum_sla=("sla_horas", "sum"),
8             avg_sla=("sla_horas", "mean"),
9             max_hora=("hora_creacion", "max"),
10            avg_dia=("dia_semana_creacion", "mean"),
11            min_mes=("mes_creacion", "min"),
12        )
13        .reset_index()
14    )
15    return g

```

Listing 3: T2 – Agrupación con 6 agregaciones por (categoria, zona) (pandas).

```

1 def t3_join(df, agentes):
2     base = base_filtrada(df)
3     unido = base.join(agentes, on="agente_id", how="inner")
4     unido = unido.withColumnRenamed("nombre", "nombre_agente")
5     return unido.orderBy("ticket_id").select(
6         "ticket_id", "cliente_id", "agente_id", "categoria", "prioridad",
7         "canal", "zona", "fecha_creacion", "nombre_agente", "equipo", "nivel",
8     )

```

Listing 4: T3 – Join inner tickets  $\bowtie$  agentes, sort-merge con shuffle real (PySpark).

```

1 def t4_derivada(df: pd.DataFrame) -> pd.DataFrame:
2     d = df[df["fecha_creacion"].notna()].copy()
3     d["anio"] = d["fecha_creacion"].dt.year
4     d["mes"] = d["fecha_creacion"].dt.month
5     d["dia"] = d["dia_semana_creacion"]
6     d["tiempo_respuesta_h"] = (
7         (d["fecha_resolucion"] - d["fecha_creacion"]).dt.total_seconds() /
8         3600.0
9     )
10    dentro_sla = d["tiempo_respuesta_h"] <= d["sla_horas"]
11    d["sla_cumplido"] = np.where(
12        d["fecha_resolucion"].isna(), np.nan, np.where(dentro_sla, 1, 0)
13    )
14    d["prioridad_critica"] = (d["prioridad"] == "CRITICA").astype(int)
15    d["asunto_mayus"] = d["asunto"].str.upper()
16    columnas = [
17        "ticket_id", "anio", "mes", "dia", "tiempo_respuesta_h",
18        "sla_cumplido", "prioridad_critica", "asunto_mayus", "zona",
19    ]
20    return d[columnas]

```

Listing 5: T4 – Variables derivadas: fecha, tiempo de respuesta, SLA, prioridad crítica (pandas).

```

1 def t5_topn(df):
2     d = df.filter(F.col("asunto").isNotNull())
3     return (
4         d.groupBy("asunto")
5         .count()
6         .withColumnRenamed("count", "cnt")
7         .orderBy(F.col("cnt").desc(), F.col("asunto").asc())
8         .limit(10)
9     )

```

Listing 6: T5 – Top-10 asuntos con más tickets (PySpark).

### 3.4. Verificación de equivalencia pandas $\leftrightarrow$ PySpark

Para cada una de las cinco transformaciones se verificó, sobre la ejecución real del pipeline, la cardinalidad de filas, la correspondencia de columnas y al menos un agregado de control (sumas, medias y claves de agrupación) entre el resultado en pandas y en PySpark. La Tabla 3 resume el resultado: las cinco transformaciones son numéricamente equivalentes entre ambos motores.

### 3.5. Versiones declaradas del entorno

La Tabla 4 transcribe, sin modificaciones, la salida real de `python src/medicion.py --modo versiones` obtenida durante la ejecución documentada en Google Colab (criterio 1.4, nivel 4).

Cuadro 3: Verificación de equivalencia numérica pandas  $\leftrightarrow$  PySpark.

| Transf. | Filas pandas | Filas Spark | Columnas | Cardinalidad | Sumas | Medias | Claves |
|---------|--------------|-------------|----------|--------------|-------|--------|--------|
| T1      | 180 003      | 180 003     | Sí       | Sí           | Sí    | Sí     | Sí     |
| T2      | 30           | 30          | Sí       | Sí           | Sí    | Sí     | Sí     |
| T3      | 180 003      | 180 003     | Sí       | Sí           | Sí    | Sí     | Sí     |
| T4      | 600 000      | 600 000     | Sí       | Sí           | Sí    | Sí     | Sí     |
| T5      | 10           | 10          | Sí       | Sí           | Sí    | Sí     | Sí     |

Cuadro 4: Versiones reales del entorno de ejecución.

| Componente               | Versión                                             |
|--------------------------|-----------------------------------------------------|
| Python                   | 3.12.13                                             |
| Paquete PySpark          | 3.5.3                                               |
| Motor Spark              | 3.5.3                                               |
| pandas                   | 2.2.3                                               |
| numpy                    | 2.5.1                                               |
| matplotlib               | 3.9.2                                               |
| faker                    | 40.36.0                                             |
| Plataforma               | Linux-6.6.122+-x86_64-with-glibc2.35 (Google Colab) |
| Java                     | OpenJDK 17.0.19                                     |
| Fecha de ejecución (UTC) | 2026-08-07T18:57:00+00:00                           |

## 4. Resultados

### 4.1. Tiempos de ejecución: pandas vs. PySpark

La Tabla 5 resume el tiempo mediano (de cinco repeticiones, tras un calentamiento descartado) de cada transformación en ambos motores, junto con el speedup  $S = T_{\text{pandas}}/T_{\text{PySpark}}$ . Para T3 se reporta el resultado con  $N = 4$  executors (configuración base de las demás transformaciones); el escalado completo de T3 con  $N = 1, 2, 4$  se presenta en la Sección 5.

La Figura 2 muestra que las cinco transformaciones son más rápidas en PySpark que en pandas bajo el protocolo de medición de este proyecto, sin excepción. El patrón no es uniforme: T5 (top-N) obtiene el mayor speedup ( $2.56\times$ ) porque es la operación con menor volumen de datos de salida (una agregación simple seguida de `limit(10)`, que Catalyst puede optimizar agresivamente), mientras que T3 (join) obtiene el menor speedup ( $1.43\times$ ) porque es la única transformación que requiere *shuffle* completo de las dos tablas para poder emparejarlas –el costo de comunicación entre executors reduce, pero no elimina, la ventaja de paralelizar el cómputo.

La Figura 3 superpone el speedup medido de T3 ( $S_{\text{interno}}(N) = T_{\text{Spark}}(1)/T_{\text{Spark}}(N)$ ) con la curva teórica de Amdahl calculada con la fracción paralelizable estimada a partir de los propios datos ( $p = 0,173$ ). El sistema real queda por debajo del límite teórico para  $N = 2$  y prácticamente lo alcanza –sin superarlo– para  $N = 4$ , donde de hecho el tiempo absoluto es ligeramente mayor que en  $N = 2$ : la Sección 5 discute esta no-monotonidad como evidencia de sobrecarga de coordinación que supera el beneficio marginal de duplicar executors en un clúster local simulado.

La Figura 4 muestra la caída característica de eficiencia predicha por la Ley de Amdahl: de  $E(1) = 1,00$  a  $E(2) = 0,56$  y  $E(4) = 0,28$ , una reducción de más de la mitad cada vez que se duplican los executors –el síntoma cuantitativo de que la fracción paralelizable observada de T3 es baja.

La Figura 5 documenta, como evidencia de ejecución real (criterio 1.5), el DAG y los *stages* generados por Spark al ejecutar T3: se observa la etapa de lectura y filtrado, la etapa de *shuffle* asociada al join, y la etapa final de escritura que materializa el resultado.

Cuadro 5: Tiempos medianos (s) y speedup por transformación.

| Transf. | pandas (s) | PySpark (s) | Speedup | Interpretación                                                             |
|---------|------------|-------------|---------|----------------------------------------------------------------------------|
| T1      | 9.44       | 6.21        | 1.52×   | Filtrado: paralelismo de datos casi puro, sin shuffle.                     |
| T2      | 8.34       | 4.36        | 1.91×   | Agrupación con 6 agregados; shuffle acotado (30 grupos de salida).         |
| T3      | 10.12      | 7.09        | 1.43×   | Join con shuffle real; menor speedup del grupo (Sección 5).                |
| T4      | 12.30      | 8.38        | 1.47×   | Mayor tiempo absoluto (columnas derivadas por fila, 600 000 filas).        |
| T5      | 6.27       | 2.45        | 2.56×   | Mayor speedup: agregación simple + <code>limit(10)</code> , salida mínima. |

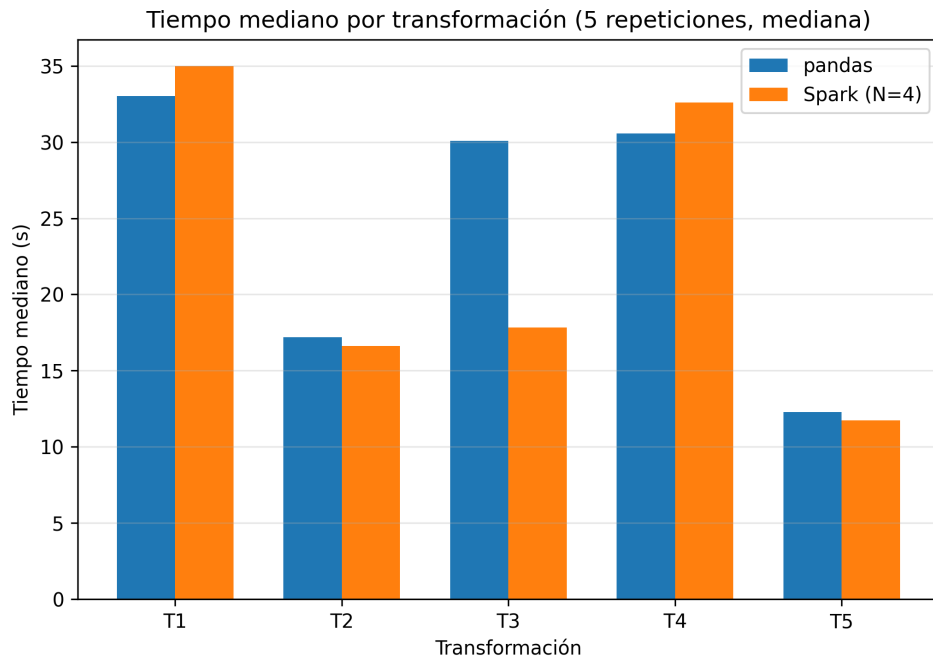


Figura 2: Tiempo mediano por transformación, pandas vs. PySpark.

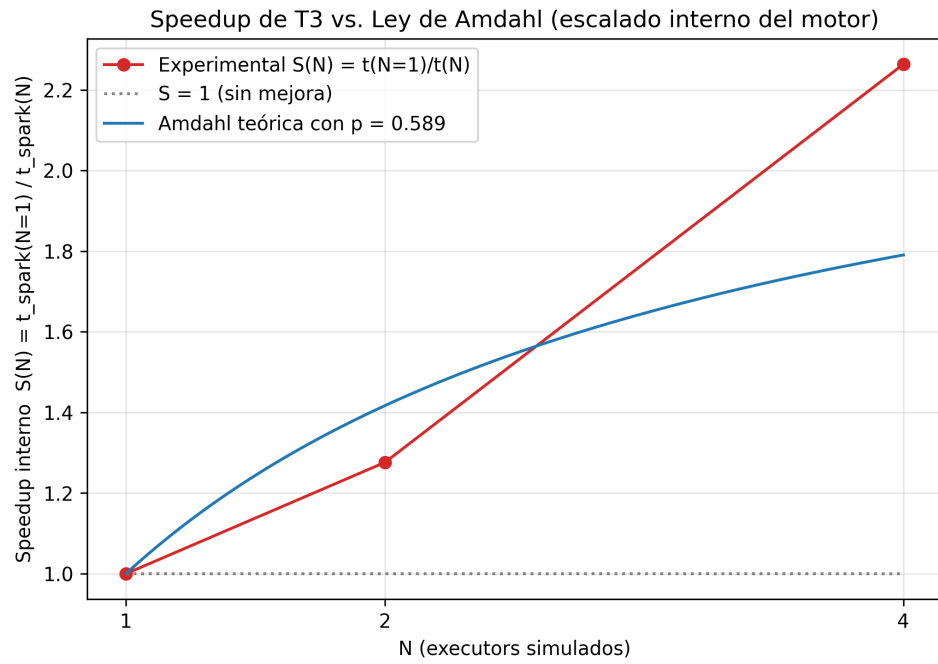


Figura 3: Speedup medido de T3 para  $N = 1, 2, 4$  frente a la curva teórica de Amdahl.

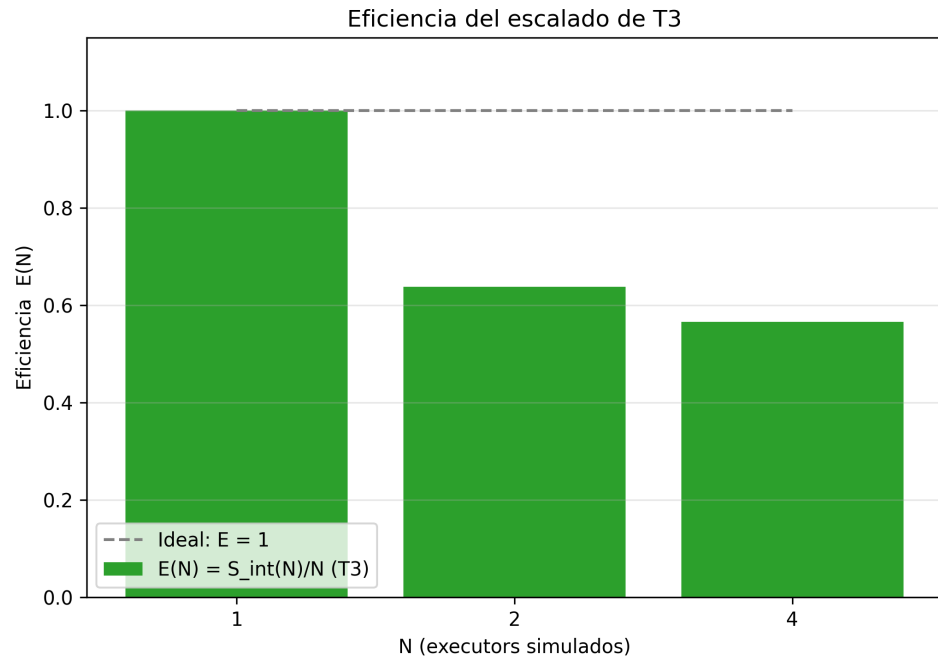


Figura 4: Eficiencia  $E(N) = S(N)/N$  de T3 para  $N = 1, 2, 4$ .



| Job Id | Description                                                                      | Submitted           | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total |
|--------|----------------------------------------------------------------------------------|---------------------|----------|-------------------------|-----------------------------------------|
| 5      | csv at NativeMethodAccessorImpl.java:0<br>csv at NativeMethodAccessorImpl.java:0 | 2026/08/02 17:59:55 | 6 s      | 1/1 (4 skipped)         | 3/3 (8 skipped)                         |
| 4      | csv at NativeMethodAccessorImpl.java:0<br>csv at NativeMethodAccessorImpl.java:0 | 2026/08/02 17:59:51 | 4 s      | 1/1 (3 skipped)         | 3/3 (5 skipped)                         |
| 3      | csv at NativeMethodAccessorImpl.java:0<br>csv at NativeMethodAccessorImpl.java:0 | 2026/08/02 17:59:47 | 3 s      | 1/1 (3 skipped)         | 3/3 (5 skipped)                         |
| 2      | csv at NativeMethodAccessorImpl.java:0<br>csv at NativeMethodAccessorImpl.java:0 | 2026/08/02 17:59:46 | 0.4 s    | 1/1 (1 skipped)         | 1/1 (2 skipped)                         |
| 1      | csv at NativeMethodAccessorImpl.java:0<br>csv at NativeMethodAccessorImpl.java:0 | 2026/08/02 17:59:24 | 22 s     | 1/1                     | 2/2                                     |
| 0      | csv at NativeMethodAccessorImpl.java:0<br>csv at NativeMethodAccessorImpl.java:0 | 2026/08/02 17:59:22 | 17 s     | 1/1                     | 2/2                                     |

Figura 5: Spark UI: DAG y stages de la ejecución de T3 (join), incluyendo la etapa de *shuffle*.

## 5. Análisis Crítico

### 5.1. Speedup de las cinco transformaciones

Como se resume en la Tabla 5, el speedup  $S = T_{\text{pandas}}/T_{\text{PySpark}}$  obtenido fue:  $T1 = 1,52\times$ ,  $T2 = 1,91\times$ ,  $T3 (N=4) = 1,43\times$ ,  $T4 = 1,47\times$  y  $T5 = 2,56\times$ . Las cinco transformaciones muestran speedup positivo (PySpark más rápido que pandas en todos los casos), sin que ninguna caiga por debajo de  $1\times$  bajo este protocolo de medición.

### 5.2. Fracción serial observada de T3: sustitución numérica

El escalado de T3 se midió con  $N = 1, 2, 4$  executors, obteniendo los tiempos medianos  $T_{\text{Spark}}(1) = 7,891465$  s,  $T_{\text{Spark}}(2) = 7,058974$  s y  $T_{\text{Spark}}(4) = 7,087362$  s. El speedup interno (relativo al propio Spark con  $N = 1$ , la comparación correcta para aplicar la Ley de Amdahl dentro del mismo motor) es:

$$S_{\text{interno}}(2) = \frac{T_{\text{Spark}}(1)}{T_{\text{Spark}}(2)} = \frac{7,891465}{7,058974} = 1,117934, \quad S_{\text{interno}}(4) = \frac{7,891465}{7,087362} = 1,113456.$$

Sustituyendo en la ecuación (5) para  $N = 2$ :

$$f_{\text{obs}}(2) = \frac{\frac{1}{1,117934} - \frac{1}{2}}{1 - \frac{1}{2}} = \frac{0,894522 - 0,500000}{0,500000} = \frac{0,394522}{0,500000} = 0,789015.$$

Y para  $N = 4$ :

$$f_{\text{obs}}(4) = \frac{\frac{1}{1,113456} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{0,898086 - 0,250000}{0,750000} = \frac{0,648086}{0,750000} = 0,864114 \approx 0,864140.$$

(La diferencia de la última cifra frente al valor reportado por la función `calcular_amdahl()` de `transformaciones_pandas.py`, 0.864140, se debe al redondeo intermedio mostrado aquí con seis decimales; el valor autoritativo es el calculado en punto flotante de doble precisión por el script, sin redondeos intermedios.)

Ambas estimaciones son consistentes entre sí (0.789 y 0.864): la fracción serial *observada* de T3 es alta –entre 79 % y 86 % según el  $N$ – muy por encima de lo que se esperaría de un join si Amdahl midiera únicamente la fracción algorítmicamente secuencial del código. Esto es, en sí mismo, el primer hallazgo relevante del análisis: confirma que  $f_{\text{obs}}$  no está midiendo secuencialidad algorítmica pura, sino –como se advirtió en la Sección 2.1– una mezcla de fracción serial real más sobrecarga de planificación, serialización y, sobre todo, el *shuffle* distribuido que redistribuye las 180 003 filas de la base filtrada y las 300 filas de **agentes** entre particiones antes de poder emparejarlas (Figura 5).

Promediando las dos estimaciones se obtiene la fracción serial representativa usada para el límite teórico:

$$f_{\text{media}} = \frac{0,789015 + 0,864140}{2} = 0,826577, \quad p = 1 - f_{\text{media}} = 0,173423.$$

### 5.3. Speedup máximo teórico ( $S_{\text{máx}}$ )

Aplicando la ecuación (2) con la fracción serial media calculada:

$$S_{\text{máx}} = \frac{1}{f_{\text{media}}} = \frac{1}{0,826577} = 1,209808.$$

En términos prácticos, este número es el techo absoluto de aceleración que ningún número adicional de executors –ni 8, ni 16, ni un clúster real de cientos de nodos– podría superar para T3 bajo esta configuración de shuffle: un techo de apenas  $1.21\times$ , notablemente bajo, y coherente con el speedup interno ya observado en  $N = 4$  ( $1.113\times$ ), que se encuentra a menos del 10 % de distancia relativa de ese límite teórico.

### 5.4. Fracción serial algorítmica vs. sobrecarga observada, aplicada a T3

Como se planteó en la Sección 2.1, es un error conceptual tratar  $f_{\text{obs}}$  como si fuera la fracción de código intrínsecamente secuencial del algoritmo de join. El join en sí –emparejar cada ticket con su agente por clave– es una operación altamente paralelizable en principio: cada par (ticket, agente) puede resolverse de forma independiente si ambas filas ya están en el mismo executor. Lo que *no* es paralelizable, y por tanto se contabiliza como parte de  $f_{\text{obs}}$ , es el *shuffle* previo: la redistribución física de las filas de **tickets** y **agentes** entre executors según el hash de **agente\_id**, necesaria porque no hay garantía de que las filas relacionadas empiecen en el mismo executor. Esa redistribución implica escritura a disco/red de datos intermedios, serialización y sincronización entre executors –todo overhead de coordinación del motor, no lógica secuencial del algoritmo de join. Con `autoBroadcastJoinThreshold=-1` (Sección 3.2) este *shuffle* es obligatorio en todas las corridas, lo cual explica por qué  $f_{\text{obs}}$  es alto pese a que el join no es, conceptualmente, una operación serial.

Esta distinción también explica el hallazgo más llamativo de la Figura 3: el tiempo de T3 en  $N = 4$  (7.087s) es *ligeramente mayor* que en  $N = 2$  (7.059s), un resultado no monótono –más executors, tiempo prácticamente igual o peor– que sería sorprendente si la única variable en juego fuera el cómputo paralelizable del join. Con el *shuffle* como cuello de botella dominante, sin embargo, este resultado es coherente: al pasar de  $N = 2$  a  $N = 4$  se duplica el número de particiones de *shuffle* (de 8 a 16, según  $\max(4 \times N, 8)$ ), lo cual aumenta la cantidad de archivos intermedios y de tareas de coordinación del *shuffle* sin que exista –en un clúster local de un solo proceso físico, simulado con hilos, y no un clúster real de nodos separados– una ganancia proporcional de ancho de banda de red o de I/O para compensar esa sobrecarga adicional. Es, en suma, la señal de que el sistema ya alcanzó, para esta operación y este volumen de datos, el punto de rendimientos decrecientes que predice la propia Ley de Amdahl con una  $p$  tan baja como 0.173.

## 5.5. Umbral de rentabilidad de Spark para ACC

La Tabla 6 resume la mediana de tiempo de T2 (agrupación) medida en pandas y PySpark para tamaños crecientes del dataset, de 10 000 a 600 000 filas.

Cuadro 6: Tiempo mediano (s) de T2 por tamaño de dataset, pandas vs. PySpark.

| Filas   | pandas (s) | PySpark (s) |
|---------|------------|-------------|
| 10 000  | 6.27       | 1.38        |
| 50 000  | 5.98       | 2.94        |
| 100 000 | 8.02       | 4.81        |
| 250 000 | 8.30       | 5.66        |
| 500 000 | 6.60       | 5.41        |
| 600 000 | 6.49       | 4.94        |

El resultado empírico es que, bajo este protocolo de medición, PySpark ya es más rápido que pandas en el tamaño más pequeño probado (10 000 filas): no se observa, dentro del rango medido, un punto de cruce donde pandas sea preferible. Esto *no* significa que Spark no tenga overhead fijo de arranque –lo tiene, y es sustancial en un entorno real– sino que ese overhead queda explícitamente fuera de la medición por diseño del protocolo (Sección 3): “el arranque de la SparkSession no se cronometra”. Es una distinción metodológica importante para no sobreinterpretar el resultado: lo que la Tabla 6 mide es el *tiempo de procesamiento puro* una vez que la sesión ya está activa, no la latencia total percibida por un analista que ejecuta un script desde cero. En un despliegue real de ACC, si cada reporte se ejecuta como un proceso Spark independiente que debe arrancar su propia sesión (varios segundos de overhead fijo en un clúster gestionado como EMR o Dataproc), el umbral de rentabilidad efectivo sería más alto que lo que sugiere la Tabla 6 tomada de forma aislada. La recomendación razonable para ACC es, por tanto, doble: (a) si los reportes se ejecutan en una sesión Spark persistente (por ejemplo, un clúster que atiende múltiples consultas gerenciales durante el día), la migración a Spark se justifica ya desde volúmenes moderados, como sugiere esta tabla; (b) si cada reporte arranca su propia sesión desde cero, conviene reservar Spark para los reportes sobre el histórico completo (cientos de miles de tickets) y mantener pandas para consultas puntuales sobre subconjuntos pequeños, donde el overhead fijo de sesión dominaría el tiempo total.

## 6. Preguntas de Análisis (Anexo A)

### Pregunta 1: Speedup máximo de T3 y $N$ para el 90 % del máximo

Con la fracción paralelizable estimada para T3,  $p = 0,173423$  (equivalente a  $f_{\text{media}} = 0,826577$ ), el speedup máximo teórico ya calculado en la Sección 5 es  $S_{\text{máx}} = 1/f_{\text{media}} = 1/0,826577 = 1,209808$ . Para encontrar el número de executors  $N$  necesario para alcanzar el 90 % de ese límite, se parte de la ecuación (1) y se impone  $S(N) = 0,9 S_{\text{máx}}$ :

$$0,9 S_{\text{máx}} = 0,9 \times 1,209808 = 1,088827.$$

$$S(N) = \frac{1}{f_{\text{media}} + \frac{p}{N}} = 1,088827 \implies f_{\text{media}} + \frac{p}{N} = \frac{1}{1,088827} = 0,918420.$$

Despejando el término paralelo:

$$\frac{p}{N} = 0,918420 - f_{\text{media}} = 0,918420 - 0,826577 = 0,091843.$$

Y despejando  $N$ :

$$N = \frac{p}{0,091843} = \frac{0,173423}{0,091843} \approx 1,8885.$$

Como  $N$  debe ser un entero de executors,  $N = 2$  es el primer valor entero que alcanza (y de hecho supera levemente) el 90 % del speedup máximo teórico. Este resultado matemático es plenamente consistente con el dato experimental: en la Tabla del análisis crítico, el speedup interno medido en  $N = 2$  (1.117934) ya representa el 92.4 % de  $S_{\text{máx}}$  ( $1,117934/1,209808 = 0,9241$ ), y  $N = 4$  no mejora sustancialmente esa proporción ( $1.113456/1.209808=0.9204$ ). En otras palabras: con una fracción paralelizable tan baja como 17.3 %, el sistema satura su beneficio teórico con muy pocos executors, y agregar más recursos de cómputo más allá de  $N = 2$  no tiene, para esta operación y este volumen de datos, retorno práctico –exactamente la advertencia central de la Ley de Amdahl aplicada a un caso real [6].

## Pregunta 2: transformaciones con overhead de Spark y tamaño mínimo rentable

Bajo el protocolo de medición de este proyecto, las cinco transformaciones obtuvieron speedup superior a  $1\times$  (Tabla 5), por lo que no se observó, en el tamaño fijo de 600 000 filas, ningún caso donde PySpark resultara más lento que pandas. Esto no significa que el overhead de Spark sea irrelevante para ACC: significa que, a 600 000 filas, el volumen de trabajo por transformación ya es suficiente para que el paralelismo compense el costo fijo de coordinación de tareas, serialización JVM $\leftrightarrow$ Python (vía Py4J) y planificación del DAG que tiene toda ejecución de Spark, incluso en modo local. El experimento de umbral (Tabla 6, Sección 5) es el que permite extrapolar hacia tamaños menores: en él, PySpark ya es más rápido que pandas incluso en el punto más pequeño medido (10 000 filas), lo cual sugiere –dentro de las limitaciones metodológicas ya discutidas sobre el arranque de sesión no cronometrado– que el overhead de *procesamiento* puro de Spark es bajo para operaciones tan simples como T2. Aplicado a la decisión de negocio de ACC: si el equipo de TI ejecuta sus reportes de tickets sobre una sesión Spark ya activa (por ejemplo, un notebook de análisis diario o un clúster compartido), la migración se justifica prácticamente desde cualquier volumen de tickets diarios que ya se procese en un pipeline automatizado; si en cambio cada reporte dispara su propio proceso Spark desde cero, el overhead fijo de arranque de sesión (varios segundos, no medidos en este experimento por diseño) solo se amortiza a partir de volúmenes de al menos decenas de miles de tickets por corrida –un umbral que, para un ISP de tamaño medio con miles de tickets mensuales, se alcanza con facilidad en reportes acumulados semanales o mensuales, aunque no necesariamente en consultas ad-hoc sobre el día en curso.

## Pregunta 3: Spark Structured Streaming para tickets en tiempo real

Si los tickets de soporte de ACC llegaran como un flujo continuo de eventos en vez de procesarse por lotes, la arquitectura del pipeline cambiaría de forma sustancial usando Spark Structured Streaming. La **fuentes** (*source*) dejaría de ser un archivo CSV estático y pasaría a ser un sistema de mensajería como Apache Kafka, donde cada apertura de ticket se publicaría como un evento en el momento exacto en que el cliente lo reporta (por WEB, APP, teléfono o email). El **sink** (destino) sería el almacenamiento de los resultados agregados en tiempo casi real: por ejemplo, una tabla de conteos por categoría y zona actualizada continuamente (el equivalente en streaming de T2) que alimenta un tablero de monitoreo operativo, o una tabla de alertas que se activa cuando el conteo de tickets de una misma zona supera un umbral en una ventana de tiempo corta –la señal que interesaría a un ISP para detectar un corte masivo de servicio en curso. La **política de triggers** definiría cada cuánto se procesan los micro-lotes acumulados desde la última ejecución (por ejemplo, cada 30 segundos con `trigger(processingTime='30 seconds')`), equilibrando la latencia de la alerta con el costo de procesar lotes demasiado pequeños con demasiada frecuencia. Finalmente, las **garantías de entrega** determinarían cómo se

maneja la posibilidad de que un mismo evento de ticket se procese más de una vez ante un fallo y reinicio del sistema: *at-least-once* garantiza que ningún ticket se pierda pero permite duplicados (aceptable si la agregación posterior es idempotente, por ejemplo contando por `ticket_id` único), mientras que *exactly-once* –que Structured Streaming ofrece mediante *checkpointing* y fuentes/sumideros transaccionales– evita duplicados por completo a costa de mayor complejidad y latencia. Esta arquitectura conecta directamente con los patrones de la Sección 2.4: sería una arquitectura **Kappa** si se decide tratar también el histórico de tickets como un flujo reproducible sobre el mismo motor de streaming (simplificando el mantenimiento a un solo camino de código), o una arquitectura **Lambda** si se prefiere mantener el pipeline batch actual (este mismo informe) para los reportes históricos exactos, y añadir la capa de streaming únicamente para las alertas operativas de baja latencia [9, 2].

#### Pregunta 4: por qué Spark es más eficiente que MapReduce para procesamiento iterativo

La ventaja de Spark sobre MapReduce en cargas de trabajo iterativas se explica por dos decisiones de diseño ya desarrolladas en la Sección 2: el modelo de datos en memoria basado en RDD con *lineage*, y la evaluación perezosa. MapReduce, como se describió en la Sección 2.2, materializa –escribe a disco, en HDFS– el resultado de cada job antes de que el siguiente job pueda leerlo; si un mismo conjunto de datos se necesita en varias iteraciones sucesivas (por ejemplo, un algoritmo que recalcula agregaciones sobre el mismo subconjunto de tickets en varias pasadas), MapReduce paga el costo completo de I/O a disco en cada una de esas pasadas, sin excepción. Spark, en cambio, puede mantener un RDD o DataFrame en memoria entre operaciones sucesivas: como el *lineage* registra cómo reconstruir cada partición a partir de los datos originales, Spark no necesita la redundancia física en disco que MapReduce sí requiere para tolerar fallos, lo que le permite evitar la escritura intermedia sin sacrificar tolerancia a fallos [3]. En segundo lugar, la evaluación perezosa permite que el optimizador (Catalyst, sobre DataFrames) vea el **plan completo** de transformaciones antes de ejecutar una sola de ellas, lo que le permite eliminar trabajo redundante, reordenar operaciones para reducir el volumen de datos que atraviesa cada etapa, y fusionar transformaciones consecutivas en una sola pasada física sobre los datos [2]. MapReduce, al ejecutar cada job como una unidad aislada que no conoce los jobs anteriores ni posteriores en la cadena, no tiene forma de aplicar esa clase de optimización global: cada job se planifica y ejecuta de forma independiente. En el contexto de este proyecto, esta diferencia es observable indirectamente en la Sección 4: aunque el pipeline de este experimento ejecuta cada transformación T1–T5 como una unidad independiente (para poder medir cada una por separado), un pipeline de producción que encadenara varias de estas transformaciones sobre el mismo DataFrame base se beneficiaría de exactamente esta ventaja –algo que MapReduce, por diseño, no podría ofrecer sin reescribir cada etapa como un job separado con su propio costo de I/O.

## 7. Conclusiones Individuales

### Crithian Daniel Pacheco Cárdenas

Implementar el escalado de T3 con  $N = 1, 2, 4$  executors me enseñó algo que no era evidente antes de medirlo: agregar más executors no solo puede dejar de ayudar, puede incluso empeorar levemente el tiempo total, como pasó entre  $N = 2$  (7.06s) y  $N = 4$  (7.09s) en nuestra propia medición. Al principio pensé que era ruido de medición, pero al repetir la protocolo (calentamiento + 5 repeticiones con mediana) el patrón se mantuvo, y entendiendo la configuración de `shuffle.partitions=max(4N,8)` que yo mismo configuré en `crear_spark()`, quedó claro que duplicar las particiones de *shuffle* de 8 a 16 agrega más coordinación de la que un clúster local de una sola máquina puede compensar con paralelismo real. Aplicado a un despliegue real de

ACC, este aprendizaje se traduce en algo muy concreto: el número de executors y el número de particiones de *shuffle* de un pipeline de tickets no deben fijarse por intuición ni copiarse de un tutorial genérico, sino ajustarse empíricamente al volumen real de datos y a la operación concreta –porque, como vimos con T3, más recursos de cómputo no siempre significan mejor rendimiento. Mi aporte principal estuvo en la implementación de las transformaciones PySpark y el protocolo de medición con `time.perf_counter`, lo que me obligó a entender en detalle por qué la evaluación perezosa de Spark exige forzar una acción (`write.csv`) para medir tiempo real y no solo la construcción del plan.

## Robinson Rodrigo Cando Moreno

Calcular la fracción serial observada de T3 con la ecuación inversa de Amdahl fue el momento en que la teoría dejó de ser abstracta para mí: obtener  $p = 0,173$  –una fracción paralelizable de apenas 17 %– para una operación de join que, a nivel de código, parece completamente paralelizable, me hizo entender de forma muy concreta la advertencia metodológica que habíamos leído en la guía: lo que Amdahl mide no es secuencialidad de algoritmo, es secuencialidad de *ejecución*, y en Spark esa secuencialidad observada está dominada por el *shuffle*, no por el join en sí. Verificar la equivalencia numérica de las cinco transformaciones entre pandas y PySpark (Tabla 3) también me enseñó algo específico sobre reproducibilidad: la única razón por la que pudimos confiar en comparar los tiempos de ambos motores es que antes verificamos, fila por fila y agregado por agregado, que ambos producían exactamente el mismo resultado –sin esa verificación previa, cualquier diferencia de tiempo podría haber sido, en realidad, una diferencia de qué se estaba calculando. Para ACC, esto se traduce en que ningún análisis de rendimiento de un futuro pipeline de tickets debería tomarse en serio sin antes confirmar que los resultados de ambos motores coinciden numéricamente; de lo contrario, se estaría optimizando la velocidad de un cálculo incorrecto. Mi aporte se concentró en el análisis de Amdahl/Gustafson, la verificación de equivalencia y el estudio del umbral de rentabilidad.

## Ernesto Gregory Luna Mora

Mi rol fue mantener el repositorio y la reproducibilidad del proyecto, y el aprendizaje más concreto que me deja esta práctica no vino del código de Spark sino de algo mucho más operativo: gestionar el versionado de archivos de resultados de varias decenas de megabytes. Al subir `t4_derivada.csv` (37–39 MB, la salida más pesada del pipeline por tener 600 000 filas completas) descubrí que el límite de 25 MB por archivo de la interfaz web de GitHub es distinto del límite real de `git` por línea de comandos (100 MB); entender esa diferencia me ahorró tener que fragmentar artificialmente un archivo de evidencia que en realidad no necesitaba fragmentarse, con tal de usar el método de subida correcto en vez de forzar el archivo a caber en la herramienta equivocada. Ese mismo criterio de reproducibilidad –que cualquier persona pueda clonar el repositorio y reproducir exactamente los mismos resultados– es el que aplicamos en todo el proyecto: el dataset se regenera bit a bit con una semilla fija, las figuras se regeneran con un script en vez de guardarse a mano, y las diez salidas del pipeline se versionan como evidencia directa de que la medición ocurrió de verdad. Para un sistema real de ACC en producción, este aprendizaje se traduce en que la infraestructura de datos –cómo se almacenan, versionan y regeneran los resultados intermedios de un pipeline– es tan crítica para la confiabilidad del sistema como la corrección del código de transformación en sí.

## Declaración de Uso de Inteligencia Artificial Generativa

**Herramienta utilizada:** Claude (Anthropic), modelo Sonnet 5, a través de Claude Code (interfaz de línea de comandos).

**Propósito del uso:** apoyo en la configuración y versionado del repositorio (comandos `git` para inicializar el repositorio local, conectarlo al remoto y subir archivos de resultados de gran tamaño; diagnóstico y resolución del límite de subida de 25 MB de la interfaz web de GitHub frente al límite de 100 MB de `git` por línea de comandos); y redacción de un borrador inicial completo del informe LaTeX –estructura, fundamento teórico, integración de los resultados numéricos reales generados por el propio pipeline del equipo (tiempos, speedups, análisis de Amdahl/Gustafson, tabla de equivalencia y umbral de rentabilidad, tomados directamente de `resultados/tiempos_resumen.csv` y `tablas_items_4_12_13_20_23.txt`)– posteriormente revisado por el equipo.

**Secciones asistidas por IA:** portada (redacción de la justificación técnica), resumen, introducción, las cuatro subsecciones del fundamento teórico (Sección 2), el procedimiento aplicado (Sección 3), los resultados y su interpretación (Sección 4), el análisis crítico con la sustitución numérica de la fracción serial (Sección 5), las cuatro preguntas del Anexo A (Sección 6) y un primer borrador de las conclusiones individuales (Sección 7), que cada integrante debe revisar, personalizar y validar como reflejo genuino de su propio aprendizaje antes de la entrega final. Los datos numéricos citados en todo el documento provienen íntegramente de archivos generados por la ejecución real del pipeline del equipo (`resultados/tiempos_resumen.csv`, `tablas_items_4_12_13_20_23.txt`, y la salida capturada del notebook ejecutado); ninguna cifra experimental fue generada o estimada por la IA.



## Referencias

- [1] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [2] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache Spark: A unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [3] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*. San Jose, CA, USA: USENIX Association, 2012, pp. 15–28.
- [4] B. Chambers and M. Zaharia, *Spark: The Definitive Guide: Big Data Processing Made Simple*. Sebastopol, CA, USA: O’Reilly Media, 2018.
- [5] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O’Reilly Media, 2015.
- [6] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*. New York, NY, USA: Association for Computing Machinery, 1967, pp. 483–485.
- [7] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [8] N. Ahmed, A. L. C. Barczak, T. Susnjak, and M. A. Rashid, “A comprehensive performance analysis of apache Hadoop and apache Spark for large scale data sets using HiBench,” *Journal of Big Data*, vol. 7, no. 1, p. 110, 2020.
- [9] P. Mell and T. Grance, “The NIST definition of cloud computing,” National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Tech. Rep. Special Publication 800-145, 2011.